Python: map() and filter() - Problem Solving Set

Level: Hard | 10 Problems | Total Marks: 70

Instructions

- Write your own Python code for each problem. Only a problem statement and a sample are given.
- map() and filter() must be the core of every solution. Use lambda where it helps.
- Not allowed: decorators, iterators or generators, and anything beyond higher-order functions.
- Wrap results in list() when you need to print them. Check your code against the sample, then try your own edge cases.
- Marks reflect difficulty. Marks are awarded for correct output, handling edge cases, and clean design.

Problem 1: Dirty Data Squares [6 marks]

A list contains integers, strings and None values. Some strings are valid whole numbers (possibly with surrounding spaces and an optional minus sign); others are junk such as 'x', '7.5' or an empty string. Build a solution that returns the squares of only the valid integer values, converting numeric strings first, and keeps the original order.

Sample

```
Input : [3, '4', 'x', None, '  5 ', -2, '7.5', '-8', '']
Output: [9, 16, 25, 4, 64]
```

Problem 2: Honour Roll [7 marks]

You are given a list of (name, marks_list) pairs. Some students have an empty marks list. Build a solution that produces (name, average) pairs only for students whose average is 60 or more, with the average rounded to 1 decimal place, ordered from the highest average to the lowest. A student with no marks must never cause a crash.

Sample

```
Input : [('Asha',[70,80,90]), ('Ravi',[40,50]), ('Meera',[]),
         ('John',[60,61,59,100]), ('Tina',[59,60])]
Output: [('Asha', 80.0), ('John', 70.0)]
```

Problem 3: Truly Unique Words [6 marks]

Given a sentence, clean each word (remove leading and trailing punctuation and convert to lowercase). Return only the words that occur exactly once in the whole sentence, in order of appearance. Think carefully: the decision for each word depends on the full cleaned word list, not on the word alone.

Sample

```
Input : 'The cat and the hat, and THE bat.'
Output: ['cat', 'hat', 'bat']
```

Problem 4: Twin Primes [7 marks]

Build a solution that returns all twin prime pairs (p, p+2) where both numbers are prime and p lies between 1 and 99. You may not use any loop statement, and you may not hard-code a list of primes. Hint: decide how to build the candidate pairs first, then decide how a primality test can itself be written as a single function without loops.

Sample

```
Output: [(3,5), (5,7), (11,13), (17,19),
         (29,31), (41,43), (59,61), (71,73)]
```

Problem 5: Above-Average Matrix [6 marks]

Given a matrix as a list of rows (rows may have different lengths and are never empty), build a new matrix where every row keeps only the elements strictly greater than that row's own average. Rows that end up empty must be removed from the result entirely.

Sample

```
Input : [[1,2,3], [5,5,5], [10,2,3,9]]
Output: [[3], [10, 9]]
```

Problem 6: All Rules Must Pass [7 marks]

Write a function that receives a list of predicate functions and a list of data. It must return the data items that satisfy every predicate, preserving order. The function must work for any number of predicates, including zero (in which case everything passes), and must not contain a loop statement.

Sample

```
Predicates: divisible by 2, divisible by 3, greater than 10
Data      : numbers 1 to 30
Output    : [12, 18, 24, 30]
```

Problem 7: Selective Raise [7 marks]

Employees are stored as dictionaries with keys name, dept and salary. Give a 10 percent raise to employees in the IT department who earn less than 50000. Everyone else stays unchanged. Return a NEW list of dictionaries and make sure the original list and its dictionaries are not modified. Then, from the new list, return the names of employees earning more than 50000.

Sample

```
Input : [{'name':'Asha','dept':'IT','salary':48000}, {'name':'Ravi','dept':'HR','salary':45000}
         {'name':'Meera','dept':'IT','salary':52000}, {'name':'John','dept':'IT','salary':30000
New salaries : Asha 52800, Ravi 45000, Meera 52000, John 33000
Names > 50000: ['Asha', 'Meera']
```

Problem 8: Leaders in a List [8 marks]

An element is a leader if it is strictly greater than every element to its right (the last element is always a leader). Build a solution that returns the leaders in their original order. You may not use any loop statement, and you may not modify the list. Hint: since each decision depends on the position, think about what you should be filtering.

Sample

```
Input : [16, 17, 4, 3, 5, 2]
Output: [17, 5, 2]
```

Problem 9: Anagram Finder [6 marks]

Given a target word and a list of words, return the words that are anagrams of the target. Ignore case when comparing, but return each word with its original spelling. A word that is identical to the target (ignoring case) is not counted as an anagram.

Sample

```
Target: 'Listen'
Words : ['Enlist','Google','inlets','Banana','silent','listen']
Output: ['Enlist', 'inlets', 'silent']
```

Problem 10: Partition With One Call **[10 marks]**

Write a function that splits data into two lists: items for which a given predicate is true, and items for which it is false, preserving order. The catch: the predicate may be expensive or have side effects, so it must be called EXACTLY ONCE per item. Calling it twice (for example, once in each of two separate filters) is not allowed. Hint: transform the data first so that each item carries its predicate result along with it.

Sample

```
Predicate: divisible by 3   Data: numbers 1 to 10
Output   : ([3, 6, 9], [1, 2, 4, 5, 7, 8, 10])
           with exactly 10 predicate calls in total
```